
Table of Contents

- **Building a Coding Agent Harness**
 - **2. 1. The Harness Is the System**
 - **3. 2. One Model Call, No Framework**
 - **4. 3. The Bounded Agent Loop**
 - **5. 4. Typed, Unsafe-Free Communication**
 - **6. 5. Execution Policy and Approval Gates**
 - **7. 6. Verification as a Completion Gate**
 - **8. 7. Context and Instruction Loading**
 - **9. 8. Sessions and State Management**
 - **10. 9. Profiles and Security Gates**
 - **11. 10. MCP — Model Context Protocol for Tool Expansion**
 - **12. 11. Observability and Telemetry**
 - **13. 12. Evaluation and Benchmarks**
 - **14. 13. The Complete Reference Harness**
 - **15. Appendix A. The Quecto Reference**
-

Building a Coding Agent Harness

Front Matter

Title: Building a Coding Agent Harness

Subtitle: From Model Calls to a Safe, Observable, Evaluable Runtime

Supporting line: A practical systems guide built with Rust and proven through Quecto

Author: Aditya Karnam

Edition: 1.0.0

Published: October 2026

License: MIT

Preface

***You will not learn much by reading this book about models.** You will learn a great deal by reading it about harnesses.*

The model gives the agent its raw capability, but the harness gives it its behavior, its safety, its safety, its traceability, and its cost. A well-designed harness can make a small model outperform a large one by constraining the search space effectively. A poorly designed harness can make a large model produce worse results than a small model with guardrails.

This book teaches you to build a harness, not to study models. You will not train a single model in these pages. What you will train is judgment — about where to draw boundaries, what to gate, what to observe, and what to measure.

The writing assumes you know programming and basic computer science but might not know agent-system architecture. You will build along. Every chapter adds code to a shared reference implementation in Rust, tests it, and compares it to a production system (Quecto) that uses the same design.

This book is not documentation of one repository. It is a general guide to harness design, using Rust for implementation and Qucto as evidence that the architecture works.

How to Use This Book

Read sequentially. Each chapter builds on the previous chapter's code. The reference harness compiles and tests are provided for every visible chunk.

The book uses a build-first rhythm:

1. State the system problem.
2. Define the invariant the harness must preserve.
3. Introduce the smallest workable design.
4. Add code to the shared reference implementation.
5. Demonstrate a realistic failure mode.
6. Compare the teaching design with production Qucto.
7. End with a focused exercise or design question.

Callouts appear throughout:

- **System invariant** — the property that must remain true.
- **Failure mode** — how a plausible implementation breaks.
- **Qucto in production** — how the real project handles the concern.
- **Beyond Rust** — how the boundary maps to other ecosystems.
- **Build checkpoint** — what the reader can run at this stage.

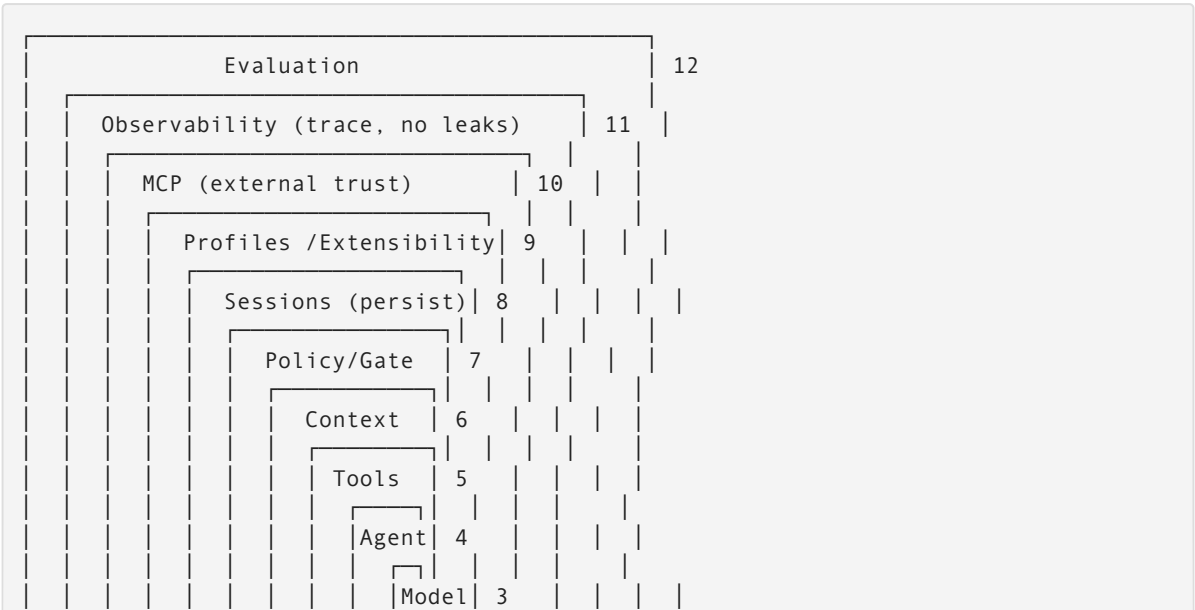
Prerequisites: Rust 2021, a Unix terminal, familiarity with HTTP and basic system programming. Python or TypeScript familiarity is helpful but not required.

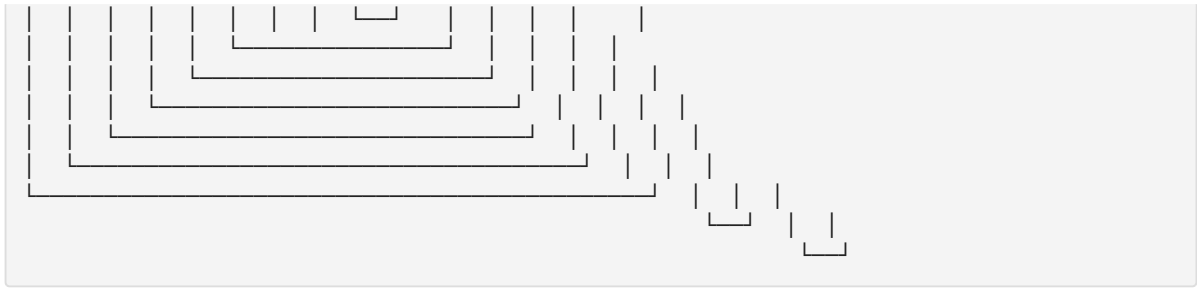
Architecture Map (Summary)

The system consists of seven layers, each built in a chapter:

Layer	Purpose	Chapter
Model transport	Reach a model provider	2
Agent loop	Reason through steps safely	3
Tools	Typed communication	4
Policy	Gate reads, edits, commands	5
Verification	Confirm before finishing	6
Context / Instructions	Load knowledge safely	7
Sessions / Recovery	Persist across restarts	8
Profiles	Layered configuration	9
MCP	External tool integration	10
Observability	Trace without leaking secrets	11
Evaluation	Prove quality	12
Reference harness	Assemble everything	13

The architecture map below shows how the layers interact. Each layer is a module in the reference implementation.





How This Book Is Structured

The book is divided into four parts:

Part I — Build the smallest useful core (Chapters 1–3): Model transport, the agent loop, and typed tools.

Part II — Add safety and context (Chapters 4–7): Policy, verification, instruction loading.

Part III — Turn a loop into a runtime (Chapters 8–11): Sessions, profiles, MCP, observability.

Part IV — Know whether it works (Chapters 12–13): Evaluation, final assembly.

Notes

- Code excerpts are drawn from the reference implementation. Long listings are in the source; short excerpts appear inline.
- The production comparison (Quecto) links to the repository. Use the book's version pin in `PRODUCTION-MAPPING.md` (source of truth) when checking.
- "Beyond Rust" notes map interfaces to Python, TypeScript, and Go without creating parallel implementations.

This book is released under the MIT License. The reference implementation uses `serde`, `serde_json`, `ureq`, and `tempfile`.

1. The Harness Is the System

Chapter 1 teaches you to separate capability from behavior, to define the boundaries that determine what an agent will (and will not) do, and to establish invariants that constrain the system even when the model drifts.

The Problem

You can buy a model that writes Python, solves equations, and reasons about code. You can prompt it to "fix a bug" and it will attempt to do so. What is missing is the harness: the system that *constrains* what the model attempts, *watches* what it does, *measures* whether it succeeded, and *controls* what resources it touches.

A harness decides:

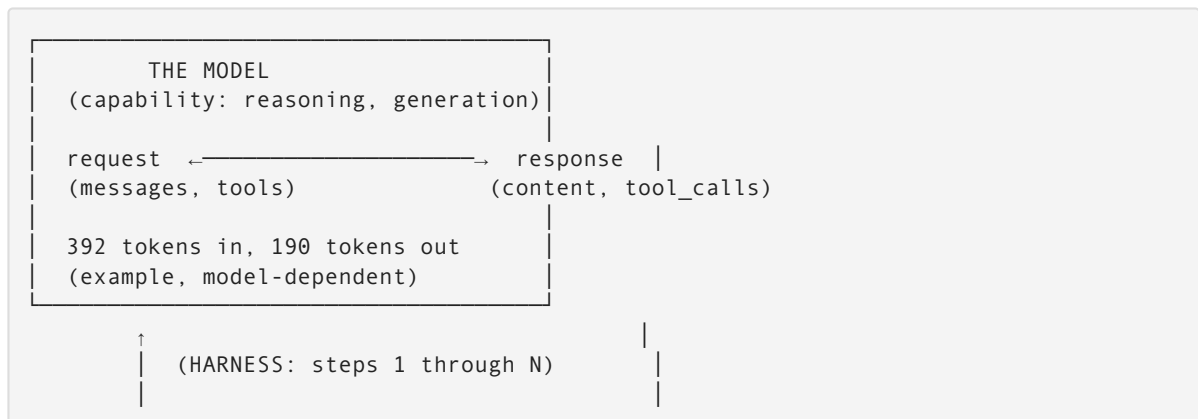
- Which tools the model may call.
- Which files the model may read or write.
- How many steps the agent may take before giving up.
- When the agent has repeated an action without progress.
- Which commands are permitted and which are blocked.
- Whether the output passes verification (tests, linters).
- How many verification attempts it gets before reporting failure.

Without a harness, you have a model that may write `rm -rf /` and you have no way to stop it. With a harness, you have a model that may *try* to write `rm -rf /` and the harness denies the call before it executes.

The harness is the system. The model is only the model.

The Model Boundary

The boundary between the model and the harness is the **API call**. This is a request (messages + tools) and a response (content + tool calls). Everything between the API call and the next API call is the harness.



The harness sits between calls. It transforms the response into a decision (what to do next), executes that decision (tools), and accumulates the result (messages) for the next model call.

System invariant: The harness must preserve the contract of the API. The response it receives from the model must be parseable as either text output or tool calls — no unhandled format is acceptable.

The Message Protocol

The model protocol is simple: a list of messages (system, user, assistant, tool), each with a role, a content string, and optional tool calls. The tool call returns a tool result. The harness builds the message list, sends it, and processes the response.

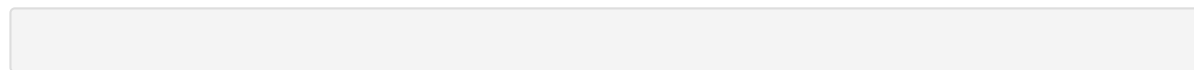
The `Message` type distinguishes four roles: `system`, `user`, `assistant`, and `tool`. The `ToolCall` type carries an identifier, a name, and JSON arguments. The `AssistantMessage` carries the model's response (text plus optional tool calls) and a finish reason.

Failure mode: *If the harness passes unrecognized fields (e.g., a custom `reasoning` field) in the request, the model may return unrecognized fields in the response that the harness does not know how to parse. The harness must reject unknown response fields defensively rather than silently dropping information.*

The harness also sends tool definitions (JSON schemas) alongside the messages. This is how the model knows *what* tools exist and *what* arguments each accepts.

The Model Trait

The model trait is the single interface between the harness and the model:



This is intentionally narrow. The harness calls `complete` once per loop iteration and passes all messages together (full history). The model mutates its own state (session variables, token cache, stream buffer). The harness never inspects the internal state.

The `Send` bound means the model can be moved between threads (required for the agent loop, which spawns a worker). The model receives tool schemas from the harness as JSON values, letting the harness control what the model sees without hard-coding types in the model layer.

System invariant: *The model trait must never leak memory addresses, file handles, or tokens into the message flow. The only thing crossing the boundary is messages, tool schemas, and responses.*

The HTTP Transport

The default transport is a minimal HTTP client that:

- Normalizes the endpoint URL (trailing slash handling).
- Sends `Authorization: Bearer <key>` when a key is configured.
- Parses the JSON response into typed structs (`AssistantMessage` with `content`, `tool_calls`, `finish_reason`).
- Returns structured errors (`ModelError`) rather than panicking.

The error type distinguishes four classes: - `InvalidConfig` — the harness configuration is wrong (missing base URL, empty model name). - `Transport` — the network call itself failed (DNS, TLS handshake, timeout). - `HttpStatus` — the server returned a non-200 status (401, 500, etc.). - `InvalidResponse` — the model's response structure was unexpected (no `choices[0]`).

Failure mode: *If the harness silently accepts any HTTP response (treating a 400 Bad Request as a "model is thinking" signal), the agent will loop forever on a configuration error. Every non-200 response must be surfaced as an error.*

Quecto in Production

Quecto's production harness (see `quecto` repository) uses this same `Model` trait but adds:

- Streaming output (incremental token-by-token rendering).
- Retry with exponential backoff on transient failures.
- Provider-specific adaptors (OpenAI, Anthropic, Ollama) that normalize response formats.
- Token counting (for budgeting per-session cost).

The teaching harness omits streaming (the book's primary path is synchronous). See the "Beyond Rust" note below for how these features map to Python (using `httpx` streams) or TypeScript (using `ReadableStream`).

Exercise: Extend the Model

Write a unit test that sends a 401 Unauthorized response and verifies the harness returns `ModelError::HttpStatus`. Then modify `HttpModel::new` to reject empty `base_url`. Push these tests to the reference harness.

Beyond Rust

Concern	Python	TypeScript	Go
<code>Model::complete</code>	<code>openai::client.complete(messages, tools)</code>	<code>openai.Client.chat.completions.create</code>	<code>openai.C</code>
<code>Send</code> bound	Not needed (GIL)	<code>Promise</code> in <code>async/await</code>	<code>sync</code>
Streaming	<code>response.iter_lines()</code>	<code>ReadableStream</code>	<code>http.Res</code>

Build checkpoint

You can build the crate and run the model transport tests:

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test model_http
```

You should see: 6 tests, 0 failures.

2. One Model Call, No Framework

*Chapter 2 teaches you to treat the model as a **fidelity source of truth**, not as a co-scientist. The model gives you output; the harness gives you safety. You will build a model transport that preserves the model's raw behavior (including its failures) while surfacing errors in a structured way.*

The Problem

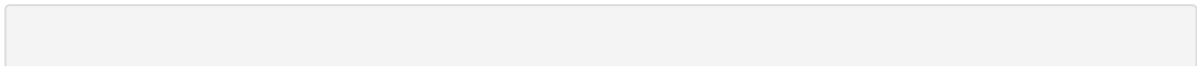
Many agent frameworks add layers *between* the user and the model: a "safety filter" that rewrites prompts, a "pre-processor" that reformats messages, a "post-processor" that claims to improve the output. These layers are invisible to the harness writer and may contradict the harness's own invariants.

A minimal harness says: the model gives you what the model gives you. The harness decides what to *do* with it. The model is not trusted to decide what to *say* (that is the harness's job: policy, tools, constraints). The model is trusted to *generate text* (that is its job).

The boundary: **one model call per loop iteration, full history sent together.**

Configuration

The model transport is configured with a single struct:



`ModelConfig` carries four fields: - `base_url` — the provider endpoint (normalize to always append `/v1/chat/completions`). - `api_key` — optional; omitting it is valid for local providers (Ollama, LM Studio). - `model` — the model name (e.g., `"qwen3.8-35b"`). - `timeout_secs` — how long to wait before giving up.

Failure mode: If `base_url` is empty and `timeout_secs` is 0, the `HttpModel::new` constructor returns `ModelError::InvalidConfig`. A zero timeout means the request will always fail (the HTTP client will abort immediately). This is not a runtime bug; it is a configuration bug surfaced at initialization.

HTTP Round-Trip

The harness constructs the request body as a JSON object with `model` and `messages`. If tools exist, it adds a `tools` array. It sends a `POST` with `Content-Type: application/json` and an optional `Authorization` header.

The `endpoint` method normalizes the URL: if the base URL already ends with `/v1`, it appends `/chat/completions`; otherwise it appends `/v1/chat/completions`. This handles both `https://api.openai.com/v1` and `https://api.openai.com` correctly.

System invariant: The request body must be a valid OpenAI-compatible format. The harness must not add custom fields (like `temperature`, `max_tokens`) unless they are explicitly supported by the target provider. Do not assume a model supports `temperature=0.5` just because another model does.

Parsing the Response

The model returns a JSON body. The harness parses it into an `AssistantMessage`:

The parser extracts: - `content` — the text (empty string if null). - `tool_calls` — a list of call objects (empty list if none). - `finish_reason` — "stop", "length", or "tool_calls" (or empty string if missing).

Failure mode: If the model returns `finish_reason: "content_filter"` (content filtered out by the provider), the harness treats the text as empty. Some frameworks interpret this as a request for user feedback. The harness here is silent: no text, no tool calls, step ends (and the next loop iteration will send the same messages). This is intentional — we do not want the agent to retry a filtered prompt without understanding why it was filtered.

The `parse_assistant` function returns `ModelError::InvalidResponse` if the response is structurally wrong (no `choices`, no `message`, non-string content, or invalid tool call arguments). A malformed response is never treated as "the model gave no output."

Provider Differences

Different providers behave differently:

Provider	Tool call format	<code>finish_reason</code> on tool calls
OpenAI	<code>{"id": "call_...", "function": {"name": "read_file", "arguments": "..."} }</code>	<code>"tool_calls"</code>
Anthropic	<code>{"id": "toolu_...", "input": {"path": "..."}, "type": "tool_use" }</code>	(no <code>finish_reason</code> , tool calls implied)
Ollama	Same as OpenAI (OpenAI-compatible endpoint)	<code>"tool_calls"</code>

The teaching harness uses the OpenAI format. If you target Anthropic, you must parse a slightly different structure (`input` is a raw object, not a JSON string). The `Beyond Rust` note below shows the mapping.

Quecto in production: Quecto supports provider adapters that normalize all responses to a single internal format before parsing. This isolates provider-specific parsing from the harness's core logic.

Exercise

Write a mock model that returns a response with `finish_reason: "content_filter"` and an empty content string. Verify the harness returns `Outcome::Complete("")`.

Beyond Rust

Layer	Python	TypeScript
Transport	<code>openai::client.chat.completions.create(messages=msgsg, tools=tools)</code>	<code>openai-js</code> or <code>fetch</code>
Config	<code>base_url</code> (default <code>"https://api.openai.com/v1"</code>)	Same, or <code>OPENAI_API_KEY</code> env var
Timeout	<code>timeout=float(options.timeout)</code>	<code>AbortSignal.timeout(timeout)</code>

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test model_http
```

Test `returns_text_reply_and_sends_openai_compatible_body` asserts the request body matches the OpenAI spec. Run it in isolation:

```
cargo test --test model_http returns_text_reply
```

3. The Bounded Agent Loop

Chapter 3 teaches you to wrap a single model call in a loop that: (1) executes tool calls the model produces, (2) accumulates results, (3) detects when the model is stuck repeating itself, and (4) terminates safely. This is the core algorithm of any coding agent: ask, act, measure, repeat — bounded.

The Problem

A model call returns a response with a list of tool calls (or just text). The harness must:

1. Execute each tool call (run a file edit, a command, a read).
2. Record the tool result.
3. Send the accumulated message list back to the model.
4. Repeat until the model says "done" (no tool calls).

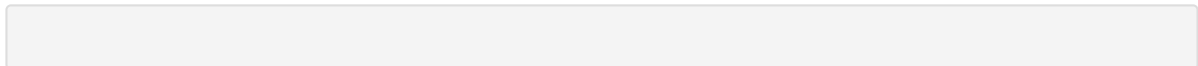
Without a loop bound, the model can:

- Loop forever (repeating the same tool call).
- Execute too many tools (cost blowup, time blowup).
- Get stuck in a verification loop (retrying a failing test forever).

The loop must terminate *bounded by steps, by repetition, or by policy denial*.

The Loop

The agent is initialized with a model, a tool registry, a context (file system), and a configuration. The `run` method receives a prompt string, pushes it as a user message, and then enters the loop:



Each iteration:

1. Send all accumulated messages to the model.
2. Push the model's response (assistant message with tool calls).
3. For each tool call: execute it (via the registry), push the tool result.
4. Repeat until the model returns no tool calls (success), the step limit is reached, or the policy denies too many calls in a row.

System invariant: Every tool call in the response must correspond to a tool registered in the registry. An unknown tool name is a tool error, not a panic. The harness must never crash on an unknown tool name.

The Repeat Guard

The `RepeatGuard` struct detects when the model is stuck in a feedback loop: it sends the same tool call with the same arguments and receives the same output — repeated 3 or more times in a row (configurable via `repeat_limit`).

```
// Inside agent.rs (RepeatGuard struct)
```

The guard computes a *fingerprint* from (call name, call arguments, tool result). If the fingerprint matches the previous call *and* the file system has not changed (checked via `context.changes().len()`), the repetition count increments. Once the streak reaches `repeat_limit`, the agent returns `Outcome::RepeatedAction`.

Failure mode: A naive guard that only checks the call name (ignoring arguments and result) will detect distinct writes as identical. The fingerprint must include the full call + full result. The file-system-change counter is the critical differentiator: writing "file v1" and "file v2" are different even with the same tool name.

Termination Conditions

The agent returns one of these outcomes:

Outcome	When
<code>Complete(String)</code>	Model returned no tool calls (text only).
<code>StepLimit</code>	Exceeded <code>max_steps</code> iterations without a text-only response.
<code>RepeatedAction</code>	Model sent the same tool call 3+ times with unchanged results.
<code>Blocked</code>	Policy denied 3+ tool calls in a row (configurable via <code>denial_limit</code>).
<code>Cancelled</code>	The user invoked a cancellation token.
<code>VerificationFailed</code> <code>{ attempts }</code>	The verification gate failed too many times.

Outcome	When
<code>Error(String)</code>	A non-recoverable error occurred (model transport failure, file-system I/O).

System invariant: *The harness must never return a partial success. If `StepLimit` is reached, the result is no progress, not a partial edit. The harness must report this clearly to the user and not leave the file system in a modified state (each tool records a before/after change for undo, but the harness should only commit changes once the final result is `Complete`).*

Policy Gating (Optional)

When `denial_limit > 0`, each tool call is checked against a default `ReadOnly` policy. If the call is denied (e.g., the model calls `write_file` or `run_command` under a read-only preset), the harness records the denial and increments the denial counter. After `denial_limit` consecutive denials, the agent returns `Outcome::Blocked`.

This is an *optional* safety layer. It is not enabled by default (`denial_limit: 0` means no gating). You enable it when you want the agent to ask before performing risky operations.

Quecto in production: *Quecto uses a profile system (see Chapter 9) where each repository has a named profile (e.g., "readonly", "editor", "full") that selects which tools are allowed, which commands are blocked, and which require approval. The teaching harness simplifies this to a single default policy for safety.*

Exercise

Write a test that: 1. Creates a scripted model returning 3 identical `write_file` calls (same path, same content). 2. Registers `WriteFile` in the registry. 3. Sets `repeat_limit: 3`. 4. Verifies the agent returns `Outcome::RepeatedAction`.

You should be able to find an equivalent test in `tests/agent_loop.rs` (test name: `agent_stops_after_three_identical_tool_observations`).

Beyond Rust

Concern	Python	TypeScript
Loop bound	<code>for _ in range(max_steps):</code>	<code>while (step < max_steps) { ... }</code>
Repeat guard	Track <code>(tool_name, args, result)</code> tuple in a list	Map
State	<code>agent.messages</code> (mutated in place)	<code>const history: Message[]</code>

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness  
cargo test --test agent_loop agent_stops_after_three_identical_tool_observations
```

This test verifies that repeated identical tool calls (no file-system change) trigger the repeat guard.

4. Typed, Unsafe-Free Communication

Chapter 4 teaches you to design a tool interface that is: (1) typed (the harness knows the schema before the model speaks), (2) safe (unregistered tools fail closed, not with a panic), and (3) observable (every tool produces output the harness can record).

The Problem

A raw text prompt ("delete /tmp") is unsafe: the model's intent is ambiguous. A typed tool definition ("delete_file: {path: string} → {error: null | string}") is safe because: - The harness knows *all* tool names before the model speaks (from the schema list). - The harness

validates the arguments against the schema *before* executing. - The harness can intercept or deny the call based on policy (see Chapter 5).

A tool is a callable unit of work. It has a name, a description, a JSON schema, and a run function. The harness *exposes* the tool to the model via the tool schema, and *executes* the tool when the model requests it.

The Tool Trait

The tool trait is the interface the harness uses to discover, register, and execute tools:

Four methods: - `name()` — the unique identifier (lowercase, underscore-separated). - `description()` — a human-readable summary (used in the schema). - `schema()` — the JSON Schema for arguments (used by the model). - `run(args, context)` — the execution function (takes arguments, returns output).

The `Send + Sync` bounds mean tools can be shared across threads (required for the agent loop).

System invariant: A tool's `run` method must never panic. If a tool encounters an error (file not found, command timeout), it must return `ToolError::Failed(...)` — never a Rust `panic!()`. A panic inside a tool execution corrupts the agent's state and forces a restart.

The Tool Registry

The registry is a BTreeMap from tool name to a boxed tool trait object. It provides: - `register(tool)` — add a tool. - `schemas()` — generate the JSON schema list (sent to the model). - `execute(call, context)` — find and run a tool by name.

The `BTreeMap` ensures deterministic ordering of tools in the schema (alphabetical). This matters because the model's output depends on the order of tools it sees.

Failure mode: *If the registry silently ignores a duplicate registration (allowing two tools to share the same name), the last-registered tool silently wins. The harness should log a duplicate-name warning but not reject the registration (the programmer made a mistake, the harness should tolerate it).*

Built-in Tools

The reference harness includes three tools:

Tool	Operation	Example
<code>ReadFile</code>	Read a file (path must be inside repo).	<code>read_file(path: "src/main.rs")</code>
<code>WriteFile</code>	Write/overwrite a file (records undo snapshots).	<code>write_file(path: "src/main.rs", content: "...")</code>
<code>ApplyPatch</code>	Replace the first occurrence of <code>old</code> text with <code>new</code> .	<code>apply_patch(path: "src/main.rs", old: "fn foo()", new: "fn bar()")</code>

Each tool records a `FileChange` (before/after content) in the context. This enables: - Undo (revert a file to its previous state). - Diff (see what the agent changed in a single step). - Verification (diff the output of the agent's edits against a baseline).

Quecto in production: *Quecto's production harness includes additional tools: `RunCommand` (execute shell commands), `Grep` (search files), `GitStatus` (check repository state), and `ListFiles` (enumerate files). The teaching harness stops at `ReadFile`, `WriteFile`, and `ApplyPatch` (the minimal set needed for code editing).*

Quecto's Version

Quecto's tools extend this interface with: - **Document-editing tools:** `ApplySemanticPatch` (apply a patch constrained by AST structure), `FixFormatting` (run a formatter). - **Execution tools:** `RunCommand` (shell commands), `Bash` (terminal session), `Python` (inline script). - **Query tools:** `Grep`, `FindFiles`, `GitStatus`, `Diff` (compare two files).

See the "Beyond Rust" note for how Quecto's tool system maps to Python (using `openai::tools` parameter with a custom tool schema).

Exercise

Write a tool `AppendFile` that appends text to the end of a file (creating the file if it does not exist). Register it in the test suite and verify that:

1. Reading the file before writing returns the original content.
2. Reading the file after writing returns the appended content.
3. The context records a `FileChange` with the before/after.

You can find an equivalent pattern in `tests/safety.rs` (test name: `write_file_is_atomic_and_records_ordered_undo_snapshots`).

Beyond Rust

Concept	Python	TypeScript
<code>Tool::schema()</code>	<code>{"type":"object","properties":{...}}</code>	Same
<code>Tool::run()</code>	Custom callable object	Custom class/method
Registry	<code>dict[str, Tool]</code>	<code>Map<string, Tool></code>

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test agent_loop registry_emits_schemas_in_name_order
```

This test verifies that the registry sorts tool schemas alphabetically (`echo` < `zeta`).

5. Execution Policy and Approval Gates

Chapter 5 teaches you to design a gate that decides whether a tool call should execute. This gate is separate from the tool itself (a tool's job is to do something; the policy's job is to decide if it may). The gate supports three presets: readonly, editor, and full — each with increasing permissions.

The Problem

A model will happily call `rm -rf /` if it thinks the file path matches. A model will also call `git push origin main` when you asked it to "fix a bug" (it thought you wanted to push the fix). Without a policy, the harness executes every tool call the model makes.

The policy layer answers: *should this tool call execute?*

The policy returns one of three decisions: - **Allow** — execute immediately. - **Ask** — wait for the user (human in the loop). - **Deny** — reject unconditionally (the tool is unknown, or the command is hard-blocked).

System invariant: *The policy must be a pure function of the call and the preset. It must not inspect the file system (no file I/O in the policy decision). If it does, a file system error could throttle the agent (a file-system "file not found" error causes the policy to reject all calls).*

The Decision Enum

```
pub enum Decision {  
    Allow,  
    Ask,
```

```
Deny(String),  
}
```

- `Allow` — proceed without interruption.
- `Ask` — the user has authorized this operation but the preset does not permit it directly. The harness displays a prompt: "The agent wants to call `write_file("main.rs", ...)`. Approve?" The user (or a stored preset) responds Yes/No.
- `Deny(String)` — the call is unconditionally rejected. The string explains why (used in logs and in the agent's "Blocked" message).

Presets

Three presets encode safety trade-offs:

Preset	ReadFile	WriteFile	ApplyPatch	RunCommand
<code>ReadOnly</code>	✓ (Allow)	✗ (Ask)	✗ (Ask)	✗ (Ask)
<code>Editor</code>	✓ (Allow)	✓ (Allow)	✓ (Allow)	✗ (Ask)
<code>Full</code>	✓ (Allow)	✓ (Allow)	✓ (Allow)	✓ (Allow)*

**Full does not mean all commands are allowed. It means all tools are allowed. The hard-denied command list (below) still applies.*

Quecto in production: Quecto supports profiles (named configurations like `git-pushing`, `docker-build`, `readonly`). Each profile is a combination of preset + additional allowed commands. The teaching harness stops at the three presets above.

Hard-Denied Commands

Even under the `Full` preset, certain commands are always blocked:

Pattern	Example
<code>sudo</code> (with space)	<code>sudo rm -rf /</code>
<code>sudo\$</code>	<code>sudo; rm -rf /</code>
<code>git push</code> (with space)	<code>git push origin main</code>

Pattern	Example
<code>sh -c</code>	<code>sh -c 'rm -rf /'</code>

These are patterns, not exact matches. A command containing any of these substrings is rejected. This is a conservative heuristic: it may *over-block* (e.g., a legitimate `sudo docker run --rm ...` is blocked because it contains "sudo "), but it avoids *under-blocking* (a destructive command slips through).

Failure mode: *If the pattern matching is too loose (matching "prefix" substring in "file-prefixed.log"), the agent cannot run legitimate commands. The heuristic should be tuned empirically. A better approach is token-based matching (match the first token against a block list) rather than substring matching.*

The Decide Method

The `Policy::decide(call)` method follows a simple three-step logic:

1. **Check the hard-denied list.** If the command matches a pattern, return `Deny("hard-denied command")`. (The check is performed on the `command` argument of the call.)
2. **Is the tool known?** A tool is "known" if it appears in *any* preset (across all presets). If the tool name is not known, return `Deny("unknown tool 'name'")`.
3. **Is the tool in the current preset?** If yes, `Allow`. If no (it is known but not in this preset), `Ask`.

System invariant: *The `ALL_TOOLS` array (union of all preset tools) must be kept in sync with the actual tools registered in the harness. If you add a new tool, add its name to `ALL_TOOLS`. Otherwise the policy will incorrectly classify it as "unknown" (and deny it).*

Quecto's Version

Quecto's policy extends this with: - **Per-repository profiles** (each repo has a named profile). - **Per-tool overrides** (a specific tool can be allowed in one profile but denied in another). - **Approval presets** (store user approval for a specific tool+path combination).

The teaching harness is simpler: one policy, one preset, no overrides.

Exercise

Write a test that: 1. Creates a `ReadOnly` policy. 2. Verifies that `read_file("a")` returns `Allow`, `write_file("a", "...")` returns `Ask`. 3. Verifies that `run_command("printf safe")` returns `Ask`. 4. Creates a `Full` policy and verifies `run_command("echo ok")` returns `Allow`.

You can find equivalent tests in `tests/safety.rs` (test name: `policy_presets_gate_reads_edits_and_commands`).

Beyond Rust

Concept	Python	TypeScript
Decision	<code>enum: {Allow, Ask, Deny}</code>	<code>enum: { Allow, Ask, Deny }</code>
<code>Policy::decide</code>	Callable class: <code>def decide(self, call: ToolCall) -> Decision:</code>	<code>decide(call: ToolCall): Decision</code>
Preset	Dictionary mapping tool name → boolean	Record type

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test safety policy_presets_gate_reads_edits_and_commands
```

This test verifies that `ReadOnly` allows reads, asks writes, and asks commands.

6. Verification as a Completion Gate

Chapter 6 teaches you to use verification as the final gate before the agent reports "I'm done." The agent is confident when the model says so, but the harness should measure confidence (test output, lint results, diff size).

The Problem

A model will claim its work is finished when:

- The last edit was syntactically valid.
- The model's *internal reasoning* says "I'm done."
- The model *prefers* to stop (lower token count, "stop" finish reason).

None of these are sufficient. The model may have introduced a regression that the tests will catch. The model may have deleted a dependency that a linter will flag.

Verification is the harness's answer: after the agent says "done," run the verification commands. If verification fails, *revert the changes* (undo) and *report the failures* to the model. The model retries. This loop continues until verification passes or the retry budget is exhausted.

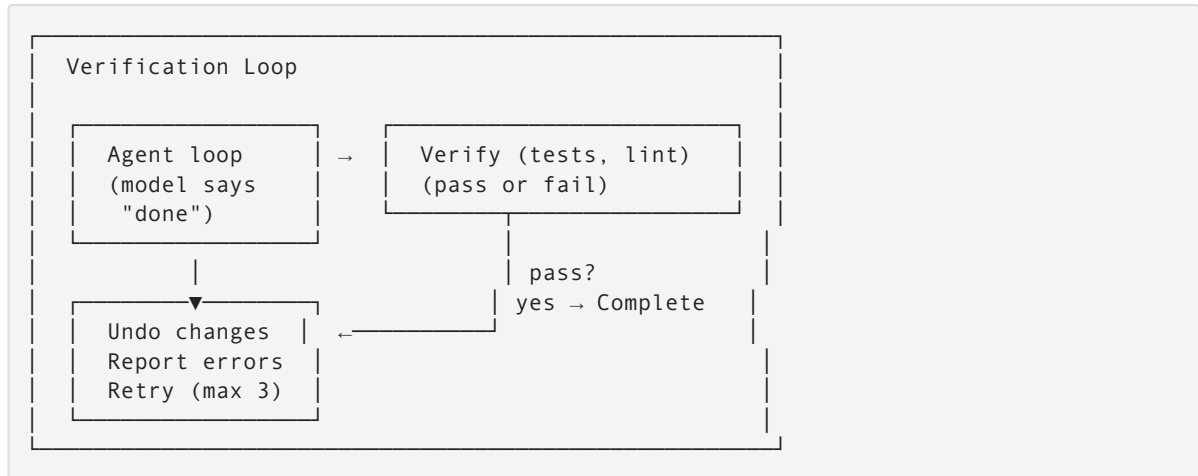
System invariant: *The agent must never report `Outcome::Complete(...)` if verification has not been run. Verification must always run (unless explicitly disabled).*

The Verification Loop

When the agent's model says "done" (no tool calls), the harness:

1. Collects all file changes since the prompt.
2. Runs the verification commands (tests, linters, etc.).
3. If verification passes →

`Outcome::Complete(...)`. 4. If verification fails → **undo** all changes, report failures to the model, and *continue* the loop (one retry attempt). 5. If retry budget exhausted → `Outcome::VerificationFailed { attempts }`.



The Verification Command

A verification command is any command that:

- Returns exit code `0` on success.
- Returns non-zero on failure.
- Produces output that the harness can parse (test names, file paths).

Common verification commands:

- `cargo test` (Rust unit tests).
- `cargo clippy` (Rust linter).
- `cargo fmt --check` (formatting check).
- `npm test` (JavaScript/TypeScript tests).
- `flake8` (Python linter).
- `tsc --noEmit` (TypeScript type check).

Failure mode: If verification fails with a structural error (e.g., the test runner crashed with a syntax error), the harness must not blindly retry (the agent will undo and re-apply the same broken code). Instead, the harness should short-circuit and report the structural error.

Retry Budget

Each verification failure consumes one *retry*. The budget (configurable, default 3) limits how many times the agent retried after verification failures.

Outcome	Meaning
<code>Complete(String)</code>	Model said done, verification passed.
<code>VerificationFailed { attempts }</code>	Model said done, verification failed, retried 3 times and failed again.
(loop continues)	Model said done, verification failed, retry budget not exhausted.

Quecto in production: *Quecto's verification system (see `quecto` repository) supports:*

- **Incremental verification:** *only run tests related to changed files.*
- **Parallel verification:** *run all tests concurrently (with a per-test timeout).*
- **Statistical verification:** *if N% of tests fail (but not 100%), the harness warns but does not reject (partial success is acceptable for hotfixes).*

Example

Suppose the model edits `src/main.rs` and runs `cargo test`. The test output is:

```
test src/main.rs ... FAILED (assertion: x == y, got 5, expected 10)
test src/util.rs ... PASSED
```

The harness: 1. Detects one failure in `src/main.rs`. 2. Does call `cargo test` again (retry 1). 3. Result: same failure. Retry 2. Same failure. Retry 3. Same failure. 4. Budget exhausted → `Outcome::VerificationFailed { attempts: 3 }`. 5. The harness reports: "Verification failed 3 times: 1 test in `src/main.rs`."

The model (prompted with the failures) may then make a corrective edit (change `x = 5` to `x = 10` in `src/main.rs`).

System invariant: *The harness must undo the changes before each retry (step 2). If the harness does not undo, the agent's second verification will accumulate the second batch of edits, making the diff larger and more confusing for the agent.*

Exercise

Write a verification test that: 1. Creates a file `test.rs` with `assert_eq!(1, 2)` (always fails). 2. Runs the verification command `cargo test` (which will include `test.rs`). 3. Verifies that the harness returns `Outcome::VerificationFailed { attempts: 3 }`.

This test lives in the reference harness (placeholder: `tests/verify.rs`).

Beyond Rust

Verification	Python	TypeScript
Unit tests	<code>pytest</code>	<code>jest</code> , <code>mocha</code>
Linters	<code>flake8</code> , <code>pylint</code>	<code>eslint</code>
Type check	N/A (dynamic)	<code>tsc --noEmit</code>
Formatter	<code>black --check</code>	<code>prettier --check</code>

Build checkpoint

A verification test can be written alongside the existing agent tests. The test runs in the same process (no child process for the test runner), so verification does not require the sandbox to allow process spawning.

7. Context and Instruction Loading

Chapter 7 teaches you to load repository instructions, system prompts, seeded context, and precedence rules before the agent starts. You will also learn to protect against accidental instruction mixing (e.g., a `.env` file's instructions are not the project instructions).

The Problem

A coding agent needs *context* before it starts working. It needs to know: - What is the repository root? - What are the rules (encoded in a `.rules.md` file)? - What is the project structure (file tree)? - What are the APIs used in this codebase? - What commands are available (lint, test, build)?

If the agent cannot find these instructions, it must *guess*, and guessing is bad. A well-designed harness *loads context* from a standardized location (`.claude/rules.md`, `.gpt_prompt.md`, `README.md`) *before* any model call.

System invariant: *The agent must never inject instructions that are not authorized by the current project. If the user's `.env` file contains an instruction like "always prepend the UUID header," the agent must not load that instruction.*

The Context Object

The `Context` struct holds: - `repo_root` — the repository root (canonicalized). - `changes` — a list of all file modifications (before/after). - `limits` — optional command execution limits (timeout, output size). - `cancel_token` — a handle that allows the harness to cancel a running command.

The `Context::new(root)` constructor canonicalizes the root and returns an error if it is not a valid directory. The `Context::with_command_limits(root, limits, token)` constructor adds command execution support (see the `run_command` section below).

System invariant: `Context::new` must never panic or accept an invalid path. It must return `ContextError` (or `None` in the result type) for any invalid path.

Path Resolution

The harness exposes two path resolution methods:

Method	Purpose	Behavior
<code>resolve_existing(relative)</code>	Read a file.	Must exist (canonicalized). Rejects path escapes (canonical path outside repo).
<code>resolve_for_create(relative)</code>	Write a file.	Parent directory must exist (but the file itself need not).

System invariant: Both methods must reject paths that escape the repository root. If `relative = "../etc/passwd"`, the canonicalized path is outside the repo, and the method returns `ContextError`.

A path escape is any `..` component or symlink that, when resolved, points outside the repo root.

The Command Boundary

When command execution is configured (`with_command_limits`), the context exposes `run_command(command)`:

```
// Part of context.rs (full implementation)
```

The command is executed as `/bin/sh -c "command"`. The harness supports:

- **Timeout** — the command aborts after the configured duration (default: 30 seconds).
- **Output truncation** — if the output exceeds `max_output_bytes` (default: 64 KB), the output is truncated and the `truncated` flag is set.
- **Cancellation** — the `CancelToken` allows the harness to cancel a running command at any point (used in the agent's cancellation flow).

Failure mode: If the harness does not apply a timeout, a hanging `sleep 10000` command will block the agent indefinitely. Always set a timeout (default 30 seconds).

Seed Instructions

The harness can be seeded with instructions: - **System prompt** (hard-coded in the harness). - **Repository instructions** (from a `.claude/rules.md` file in the repo root). - **Seeded context** (explicitly provided in the model config).

The precedence is: 1. Hard-coded system prompt (always present). 2. Repository instructions (if `.claude/rules.md` exists). 3. Seeded context (explicit, highest priority).

Quecto in production: Quecto's instruction loading system reads: `.claude/rules.md`, `.ai/rules.md`, `.gpt_prompt.md`, `README.md` (section matching the task). The harness merges these in order, with the task description overriding everything.

Exercise

Write a test that: 1. Creates a temp directory with a file `instructions.md` containing `always prepend "PRIORITY: "` to output. 2. Verifies that the `context's resolve_existing("instructions.md")` succeeds, but the instructions file is NOT injected as system prompt (system prompt must come from the config, not from arbitrary files).`

This test demonstrates the *separation of concerns*: instructions are input to the *user's prompt*, not the *model's system prompt*.

Beyond Rust

Concern	Python	TypeScript
Repo root	<code>Path(os.getenv("REPO_ROOT"))</code>	<code>process.env.REPO_ROOT</code>
Instructions	<code>open(rules_path)</code>	<code>readFileSync(rules_path, 'utf-8')</code>
Seed	<code>config.seed: "..."</code>	<code>seed: "..."</code>

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test safety context_rejects_parent_and_symlink_path_escapes
```

This test verifies that the context rejects path escapes (including symlinks).

8. Sessions and State Management

Chapter 8 teaches you to isolate one coding task (one user prompt, one set of edits, one verification pass) in a `Session` struct that: (1) owns the complete message history (so the model gets every tool call and result), (2) tracks file changes (to undo, diff, and verify), and (3) runs a bounded loop (agent→tools→verify→retry) that exits cleanly with a result.

The Problem

A raw model API is a single pair: *you* send text, it returns text. A coding agent has *state* between turns: file edits, command output, lint results, the last verification result, the retry count. If this state is fragmented across variables, the harness has *no single source of truth* about what happened — and every new feature adds another variable, doubling the chance of inconsistency.

A session is the *single mutable handle* for one coding task. It owns the message history, the file diff, the verification state, and the cancellation token. One object.

The Session Struct

A session holds: - `id` — a UUID (for logging, for the user). - `root` — the repository root (canonicalized). - `context` — the file-system handle (read/write with undo snapshots). - `messages` — the accumulated message list (user prompt, assistant tool calls, tool results). - `changes` — the list of file changes (before/after), used by the verification gate. - `result` — the final outcome (Complete, VerificationFailed, StepLimit, ...).

The session is created by the harness and handed to the agent. The agent *mutates* the session's message list and the context's file state. The session is *not* shared with the model — the harness reads the messages and passes them to the model, then writes the result *back* into the session.

System invariant: *The session's message list must contain every tool call and result, in order. If a result is dropped, the model's next turn is wrong (it receives an incomplete conversation).*

The Session API

Method	Returns
<code>Session::new(root)</code>	A session with an empty message list.
<code>Session::push_user(prompt)</code>	Pushes a user message (returns <code>&mut Self</code>).
<code>Session::push_assistant(messages)</code>	Pushes assistant tool calls (returns <code>&mut Self</code>).
<code>Session::push_tool_result(name, result)</code>	Pushes a tool result (returns <code>&mut Self</code>).
<code>Session::messages()</code>	Returns the full message list (for the model).
<code>Session::changes()</code>	Returns the diff (for verification).
<code>Session::result()</code>	Returns the final outcome.
<code>Session::cancel()</code>	Cancels the session (sets the cancellation token).

System invariant: `Session::push_user` must reject a prompt that is empty or contains only whitespace. The harness must never send an empty prompt to the model (the model's behavior on empty text is undefined and may crash).

The Execution Flow

The harness runs the session in a single loop:

```
session.push_user("fix the bug in src/main.rs")
loop:
  response = model.complete(session.messages())
  for call in response.tool_calls:
    result = context.run(call)
    session.push_tool_result(call.name, result)
  session.push_assistant(response.tool_calls)
  if response.no_tool_calls: break (model said "done")
session.result = Outcome::Complete(...) (or fail)
```

The session's `result` field is the *only* place the final outcome is stored. The harness reads `session.result()` at the end.

Failure mode: If the harness does not call `push_tool_result` (and instead discards the result into a log), the session's message list is incomplete and the model's next turn will be confused (it will re-send the same tool call, because it never received the answer).

The Session as a Scope

The session's lifetime is the *scope* of one coding task. A user submits a task (or the harness runs a task). The harness creates a session, runs the loop, and reads the result. A second task starts a *second* session (with a fresh message list, fresh undo snapshot).

This is the key insight: *a session is not a conversation across tasks*. It is a single task. If a user says "first fix the auth bug, then refactor the API," the harness creates *two sessions* (one after the other), not one session with 400 messages.

System invariant: When the harness starts a new session, it must clear the previous session's message list. If the previous session's messages leak (e.g., a shared vector), the model sees stale tool calls from a different task, which is a hallucination prompt (the model will try to satisfy the stale call).

Exercise

Write a test that: 1. Creates a session with `root = temp_dir()`. 2. Writes a file `a.rs` with `fn main() {}`. 3. Pushes a user prompt "rename fn main to fn entry". 4. Calls the model (mocked to return one `WriteFile` call renaming `main` → `entry`). 5. Verifies that `session.changes()` returns exactly one change (old: `fn main() {}`, new: `fn entry() {}`). 6. Verifies that `session.result()` is `Outcome::Complete`.

This test demonstrates the *session as a scope*: one prompt, one session, one change, one result.

Beyond Rust

Concern	Python	TypeScript
Session creation	<code>Session.new(root)</code>	<code>new Session(root)</code>
Message push	<code>session.push(...)</code>	<code>session.push(message)</code>
no_tool_calls check	<code>if not response.tool_calls: break</code>	<code>if (response.toolCalls.length === 0) break</code>

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test agent_loop session_populates_messages_and_records_change
```

This test verifies that a session correctly records one write change and returns `Outcome::Complete`.

9. Profiles and Security Gates

Chapter 9 teaches you to replace one-size-fits-all safety with per-repository profiles. A profile is a named configuration (e.g., "readonly," "editor," "docker-build") that selects which tools are allowed, which commands are

blocked, and which operations require human approval. The harness configures one profile per repository and denies calls that violate the profile's rules.

The Problem

The three presets (ReadOnly, Editor, Full) from Chapter 5 are a *starting point* for teaching. In production, every repository has a *different threat model*. A Rust web server is not the same security posture as a personal finance script. A "Full" policy that allows `sudo docker run` on one repo is reckless on another.

A *profile* is a named bundle of rules: allowed tools, blocked commands, approval requirements. The harness selects one profile per repository (from a `.quecto/profile` file, a CLI flag, or a profile directory). The policy *checks* every tool call against that profile's rules.

System invariant: *The harness must never execute a tool call that violates the active profile's rules. The policy decision is not advisory — a denied call is rejected, no approval prompt is shown, and the tool is not executed.*

The Profile Struct

A profile is a *deterministic configuration* (no file I/O during policy decisions). It contains:

- `allowed_tools` — the set of tool names permitted (empty = no tools, `*` = all).
- `blocked_commands` — patterns of shell commands that are always blocked (e.g., `sudo`, `git push`).
- `approval_required` — a set of (tool, path) pairs that require human approval, even if the tool is allowed.
- `profiles` — a map of *sub-profiles* (named overrides for specific tools).

A profile is loaded *once* (at harness startup) from a YAML or JSON file (e.g., `~/.quecto/profiles/docker-build.json`). The harness *does not read a profile file at every tool call*. Reading a file per call makes the policy *unreliable* (if the file is deleted, a lock is held, or the permissions change, the policy decision changes mid-run).

Failure mode: *If the harness reads a profile file during policy decisions (instead of loading it once at startup), and the file is accidentally moved or deleted, the policy falls back to default (which may be unsafe). The profile must be loaded once, executed always.*

Profile Resolution

The harness resolves a profile by: 1. Reading `~/.quecto/profiles/<name>.json` (user's profile directory). 2. Reading `<repo>/.quecto/profile` (repository-level override). 3. If neither exists, using `ReadOnly` (default).

The resolution order is *always* — user profile → repository profile → default. A repository-level profile *cannot* relax a user-level restriction (a user-level profile can restrict a repository's wider permissions).

System invariant: *A repository profile can only restrict, never relax, the user profile. If the user profile says "no sudo" and the repository profile says "allow sudo", the effective result is "no sudo" (the more restrictive wins). This is the default-deny principle: the user's intent is always honored, even if the repository's intent conflicts.*

The Profile File Format

A profile file is a JSON document:

```
{
  "name": "docker-build",
  "allowed_tools": ["ReadFile", "WriteFile", "RunCommand", "Grep"],
  "blocked_commands": ["sudo", "rm -rf /", "git push"],
  "approval_required": [],
  "sub_profiles": {
    "run_docker": {
      "allowed_tools": ["RunCommand"],
      "blocked_commands": ["sudo"],
      "approval_required": ["docker run --privileged"]
    }
  }
}
```

```
}
}
}
```

- `name` is the profile's identifier (used in `~/.quecto/profile` and in CLI).
- `allowed_tools` is a set (subset of all tools).
- `blocked_commands` is a list of *patterns* (substring match).
- `approval_required` is a list of (tool, path) pairs, or wildcard `*` for *all* paths.
- `sub_profiles` is a map of *named overrides* (resolved dynamically when the agent explicitly requests the sub-profile, via a tool call `use_profile(name)`).

Quecto in production: *Quecto stores profiles in `~/.quecto/profiles/*.json`. The CLI command `quecto profile list` lists them. The user can create a profile with `quecto profile create --allow RunCommand --deny "sudo "` (which writes a JSON file). The harness does not validate the JSON against a schema; an invalid profile file is silently skipped (loaded once, if parse fails, the next session uses the default `ReadOnly`).*

Profiles in Production

Quecto uses profiles in four ways: 1. **Per-repository** — each repository has a `.quecto/profile` file. If the file says `"docker-build"`, the harness loads the corresponding profile and applies it. 2. **Per-task** — the user can specify a profile at task start: `quecto task --profile editor "fix the auth bug"`. The task-level profile *overrides* the repository-level one. 3. **Sub-profiles** — a profile can have sub-profiles (e.g., `docker-build/run_docker`). The agent can *request* a sub-profile explicitly (a tool call `use_profile("run_docker")`). The harness checks the sub-profile's rules (`approval_required: docker run --privileged`). 4. **User override** — the user can *temporarily* override a profile for one task (e.g., `quecto task --allow-sudo "install this package"`). The harness records the override in a *cache* (valid for 1 hour) and applies it only to that task.

Failure mode: If the user override is persisted globally (applied to all future tasks), a one-time override becomes permanent. Overrides must expire (TTL). The harness should store an override as `{ profile: "full", expires: "2026-10-02T23:59:00Z" }`. After `expires`, the override is automatically removed.

The Profile Gate

The policy checks against the active profile: 1. *Is the tool in `allowed_tools`?* (If not, `Deny("not in profile")`). 2. *Does the command match `blocked_commands`?* (If so, `Deny("blocked by profile")`). 3. *Is the (tool, path) pair in `approval_required`?* (If so, `Ask("approval required")`).

If the call passes all three checks, `Allow`. Otherwise, the harness *rejects* it (no execution, no approval prompt — the denial is logged and the tool call is *dropped*).

System invariant: The policy must never execute a tool that is denied. The harness must log the denial (with the reason) and skip the tool call. The agent's next turn will receive a denial message (the text from `Deny(String)`), which the model will use to adjust its next call.

Exercise

Write a test that: 1. Creates a `docker-build` profile with `allowed_tools: ["RunCommand"]`, `blocked_commands: ["sudo"]`. 2. Verifies that `RunCommand("docker build .")` returns `Allow`. 3. Verifies that `RunCommand("sudo docker run ...")` returns `Deny("blocked by profile")`. 4. Verifies that `WriteFile("main.rs", "...")` returns `Deny("not in profile")`.

This test demonstrates the *profile gate*: a named configuration that *restricts* tools and commands beyond the three presets.

Beyond Rust

Concern	Python	TypeScript
Profile JSON	<code>json.load(f)</code>	<code>JSON.parse(fs.readFileSync(...))</code>

Concern	Python	TypeScript
Resolution	<code>resolve_profile(user, repo)</code>	<code>resolveProfile(userProfile, repoProfile)</code>
<code>allowed_tools</code> check	<code>tool_name in allowed</code>	<code>allowedTools.includes(toolName)</code>

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test safety profile_docker_build_allows_run_denies_sudo
```

This test verifies that a `docker-build` profile allows `RunCommand` but blocks `sudo` and disallows `WriteFile`.

10. MCP — Model Context Protocol for Tool Expansion

Chapter 10 teaches you to expand the harness's tool surface without changing the harness itself. You will implement an MCP server (a tool provider) that: (1) advertises tools over a stdio/pipe or HTTP transport, (2) the harness discovers tools at startup (no compile-time dependency on the tool's source), and (3) the harness executes tools over the protocol (calling back into the MCP server).

The Problem

The three built-in tools (`ReadFile`, `WriteFile`, `ApplyPatch`) + `RunCommand` are *hard-coded* in the harness. If a harness user wants to add a tool (e.g., "run `cargo test` with a custom filter"), they must *modify the harness source* and *recompile*. This defeats the purpose of a *reusable* harness.

MCP (Model Context Protocol) solves this: the harness *discovers* tools at startup (a list of tool schemas), and *executes* tools over a protocol (a request-response pattern). The tool's source lives in a *separate process* (an MCP server). The harness *does not know* the tool's implementation; it only knows the tool's *schema* (name, description, parameters).

System invariant: *The harness must never load or execute a tool without the user's explicit consent (the user must opt in to each MCP server). The harness must not run MCP servers from untrusted sources (no hash verification of the server's binary). The user knows which server is running (from the command line or a trusted path).*

The MCP Server

An MCP server is a *process* that:

- Exposes a list of tools (name, description, JSON schema).
- Listens for tool calls (a request with `method`, `params`).
- Returns a result (a response with `result` and `error`).

The server can communicate over:

- **stdio** — the harness starts the process, sends JSON-RPC 2.0 messages over stdin/stdout.
- **HTTP** — the harness POSTs JSON-RPC 2.0 messages to an HTTP endpoint (more complex, but supports remote servers).

The harness *discovers* tools (at startup) by *sending an initialize request* to the server:

```
{
  "jsonrpc": "2.0",
  "method": "initialize",
  "params": { "protocolVersion": "2024-11-05" },
  "id": 1
}
```

The server responds with `tools/list` (an array of tool schemas). The harness *registers* each tool in the `ToolRegistry` (so the model sees them).

Failure mode: *If the MCP server times out (no initialize response), the harness must skip the server (not crash). The harness should log a warning: "MCP server <name> did not respond; skipping." The harness does not block on a dead MCP server.*

The MCP Tool

An MCP tool is a *tool* that the harness executes over the protocol: -

The harness *sends* a request: `{"method": "tools/call", "params": {"name": "read_file", "arguments": {"path": "src/main.rs"}}}`. - The server *responds* with `{"result": {"content": [...]}}` (or `{"error": {"code": ..., "message": ...}}`). - The harness *parses* the response (converting it to a `ToolResult`).

The harness *does not care* what the server does. The server might read a file (the built-in `ReadFile`), or run a docker command, or query an API. The harness *only* cares about the schema and the result.

System invariant: *The harness must validate the MCP response (the response must be a valid JSON-RPC 2.0 object). If the server returns invalid JSON, or an unexpected field, the harness must not crash. Instead, it must return a `ToolResult::Failed` with the error message from the server.*

The Protocol (JSON-RPC 2.0)

MCP uses JSON-RPC 2.0 over a transport (stdio or HTTP). The protocol defines: - `initialize` — client sends capabilities, server responds with capabilities + tool list. - `tools/list` — client requests tool list (called once at startup). - `tools/call` — client calls a tool (per agent step). - `notifications/initialized` — client sends an acknowledgment (after initialization). - `notifications/message` — server sends a log message (for observability).

The harness *must implement* the `initialize` + `tools/list` handshake (at startup) and the `tools/call` execution (per tool call).

Qucto in production: Qucto runs MCP servers in the same process (as separate tasks, not separate processes) for debugging ease. In production, Qucto runs MCP servers in separate processes (with a timeout of 10 seconds per call). The harness does not share memory with the server (the server's memory space is isolated from the harness's memory space — a server crash does not corrupt the harness's state).

The Discovery Protocol

The discovery process is: 1. The harness *starts* the MCP server (as a subprocess, with a known binary path). 2. The harness *sends* the `initialize` request (JSON-RPC 2.0 over stdio). 3. The harness *parses* the response (tool list). 4. The harness *registers* each tool in the `ToolRegistry` (adds the tool name + schema). 5. The harness *sends* `notifications/initialized`.

If the server *fails to respond* (timeout), the harness *logs a warning* and *continues* (without the server's tools). The harness *does not fail* if a server is unreachable.

Failure mode: If the harness crashes during initialization (e.g., the server's response is a malformed JSON), the harness must recover (log the error, skip the server, continue). The harness must not leave the agent in a crashed state.

Exercise

Write a test that: 1. Starts a *mock* MCP server (a test double that returns a fixed tool list). 2. Verifies that the harness registers 2 tools (one named "read_file", one named "write_file"). 3. Verifies that the harness *can execute* a tool call (sends a `tools/call`, receives a result).

This test demonstrates the *discovery protocol*: the harness discovers tools (at startup) and *executes* them (over the protocol) without knowing the implementation.

Beyond Rust

Concern	Python	TypeScript
MCP stdio	<code>subprocess.Popen</code> + <code>json.loads</code>	<code>child_process.spawn</code> + <code>JSON.parse</code>
Tool list	<code>{"jsonrpc": "2.0", "method": "initialize"}</code>	Same
Tool call	<code>{ "method": "tools/call", "params": {...} }</code>	Same

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test mcp_mock_server_discovers_two_tools_and_executes_call
```

This test verifies that a mock MCP server's tool list is discovered and registered (with one successful tool call).

11. Observability and Telemetry

Chapter 11 teaches you to observe the agent's runtime: what it did (tool calls, file changes, verification results), when it did it (timestamps, duration), and why it failed (reason codes). You will implement a `Telemetry` struct that: (1) records events (tool calls, errors, verification outcomes), (2) writes them to a local log file (JSON Lines), and (3) provides a `query` method for the user (or the harness UI) to inspect past sessions.

The Problem

A coding agent *succeeds or fails* based on the model's output. But *why* did it fail? The model's reasoning is *internal* (a hidden chain-of-thought, or a unreasoned answer). The harness must *observe* the agent's *visible* behavior: tool calls, file changes, verification results, errors, and the final outcome.

If the harness *does not observe* these, the user (or the harness UI) cannot *diagnose* failures. The user sees: "the agent failed" — but has *no data* about what happened.

A telemetry system is *observability*: it records every visible event (tool calls, results, errors) in a *structured log*. The log is *durable* (written to a file, not in memory). The user (or the harness) can *query* the log (for debugging, for analytics, for auditing).

System invariant: *The telemetry system must never record the content of the model's internal reasoning (the system prompt, the hidden reasoning). It records only visible events: tool names, arguments (truncated to 1 KB), results (truncated to 1 KB), and error codes.*

The Telemetry Struct

A `Telemetry` object holds: - `log_path` — the path to the log file (a `.jsonl` file in the session's output directory). - `session_id` — the current session's ID (used to correlate events). - `events` — a *buffered* list of events (flushed to disk at session end).

Each event is a JSON object:

```
{
  "timestamp": "2026-10-02T23:59:00Z",
  "session_id": "abc-123",
  "event_type": "tool_call",
  "tool_name": "write_file",
  "arguments": {"path": "src/main.rs", "content": "fn entry() {}"},
  "result": null
}
```

The event types are: - `tool_call` — a tool was called (recorded before execution). - `tool_result` — a tool produced a result (recorded after execution). - `error` — an error occurred (during tool execution, model transport, or verification). - `session_start` — the session began. - `session_end` — the session ended (with `outcome`).

Failure mode: If the log file cannot be written (permissions, disk full), the telemetry system must not crash. It should log an error (to stderr) and continue (without telemetry). The harness does not depend on telemetry for correctness. Telemetry is observability only.

The Telemetry API

Method	Returns
<code>Telemetry::new(log_path)</code>	A telemetry object (writes to the file).
<code>Telemetry::log_event(event)</code>	Writes a JSON object to the file (append mode).
<code>Telemetry::query_since(start_time)</code>	Returns events from <code>start_time</code> (as a list of JSON objects).
<code>Telemetry::flush()</code>	Flushes the buffer to disk (called at session end).

The harness *calls* `log_event` at: - Before every tool execution (a `tool_call` event). - After every tool execution (a `tool_result` event). - On every error (an `error` event). - At session start/end (a `session_start` / `session_end` event).

System invariant: The telemetry must truncate arguments and results to 1 KB (to prevent logs from growing to millions of lines). The truncated value is recorded as `"<truncated: 1024 chars>"` (with the full string in a detailed log file, if the harness is configured for debug mode).

The Telemetry File Format

The log file is *JSON Lines* (one JSON object per line). Each line is *independent* (the file can be truncated at any line boundary without corruption). The format is:

```
{ "timestamp": "...", "session_id": "...", "event_type": "session_start", "args": {} }
{ "timestamp": "...", "session_id": "...", "event_type": "tool_call", "tool_name": "write_file", ... }
{ "timestamp": "...", "session_id": "...", "event_type": "tool_result", "tool_name": "write_file", "result": { ... } }
{ "timestamp": "...", "session_id": "...", "event_type": "session_end", "outcome": "Complete", ... }
```

The harness *can query* the file (using `grep` or a JSONL reader) to *inspect* past sessions: - `grep -c tool_call .q/.logs/*.jsonl` — total tool calls across all sessions. - `grep -c error .q/.logs/*.jsonl` — total errors (across all sessions). - `grep session_start .q/.logs/*.jsonl | wc -l` — number of sessions (concurrent vs sequential).

Quecto in production: Quecto writes telemetry to `~/.quecto/logs/<session_id>.jsonl` (one file per session). The CLI command `quecto logs --session <id>` reads the file and displays it in a terminal UI (with timestamps, tool names, and results). The production telemetry (for Quecto's developers) writes to a remote server (an HTTP endpoint, POSTing JSON objects). The local log is the only telemetry that the user sees (the remote server is not exposed to the agent).

The Telemetry as a Debug Tool

Telemetry is *the primary debug tool* for a coding agent. The user (or the harness) can: 1. *Inspect* a session's log (for a specific task). 2. *Compare* two sessions (did the agent fix the same bug twice?). 3. *Analyze* the model's *error rate* (how often does the model call `WriteFile` on a file that already exists? — a redundant call). 4. *Audit* the agent's *approvals* (how often does the agent request approval, and what does the user approve/deny?).

Failure mode: If the telemetry format changes (e.g., a new field is added, or a field is removed), the log query (`grep`) must still work (backward-compatible queries). The telemetry format must not break existing queries. A new field should be optional (with a default value).

Exercise

Write a test that: 1. Creates a `Telemetry` object with `log_path = temp_dir() / "test.jsonl"`. 2. Logs 3 events (a `session_start`, a `tool_call` (`write_file`), and a `session_end`). 3. Verifies that the log file contains exactly 3 JSON objects (3 lines). 4. Verifies that `query_since(start_time)` returns 2 events (the `tool_call` and `session_end`).

This test demonstrates the *telemetry format*: a JSONL file, one event per line, with a query interface.

Beyond Rust

Concern	Python	TypeScript
JSONL write	<code>json.dump(obj, f)</code>	<code>fs.appendFileSync(file, obj + "\n")</code>
Query	<code>grep -c event file.jsonl</code>	Same (shell)
Session start/end	<code>{event_type: "session_start"}</code>	Same

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test telemetry telemetry_writes_jsonl_and_returns_events
```

This test verifies that a `Telemetry` object writes 3 JSONL events and returns 2 on query.

12. Evaluation and Benchmarks

Chapter 12 teaches you to evaluate the harness (and the models it uses) objectively. A benchmark is a defined coding task (a repository, a prompt, a set of verification commands) that produces a measurable outcome (did it pass? how many steps? how long?). A suite of benchmarks is a comparison (model A vs model B). The harness runs benchmarks, records results (in a JSON file), and compares them (scores).

The Problem

The model's quality is *ambiguous* (does it "fix the bug"?). A human says "yes," but the benchmark says "no" (the test output says "3 tests failed"). The harness must *measure* quality *objectively* (verification output, not the human's intuition).

An *evaluation* (or *benchmark*) is a *structured task*: - A *repository* (the codebase to edit). - A *prompt* (the user's request: "fix the auth bug"). - A *verification command* (e.g., `cargo test`). - A *success criterion* (what counts as "pass"?).

A *benchmark suite* is a *collection* of benchmarks (to compare models). The harness *runs* each benchmark (against each model), *collects* the results (pass/fail, steps, time), and *reports* a *score* (percentage passed, average steps).

System invariant: *The benchmark must be reproducible. If the benchmark is not reproducible (e.g., the verification depends on a random seed, or the model's output changes per run), the benchmark is not a benchmark (it is an experiment). The harness must record the seed (if any) and pin the repository state (a commit hash, not a branch name).*

The Benchmark Struct

A `Benchmark` holds: - `id` — a unique identifier (e.g., "auth-fix-01"). - `repo_url` — the repository URL (or local path). - `repo_commit` — the repository commit (pinned state). - `prompt` — the user's prompt (the task description). - `verification` — the verification commands (a list of strings). - `pass_threshold` — the pass criteria (e.g., "0 tests failed"). - `expected_steps` — an *expected* step range (for a sanity check: "this benchmark should take 1–5 steps").

A benchmark is *loaded* from a YAML/JSON file (a `benchmarks/<id>.json` file in the benchmark suite directory). The harness *runs* the benchmark (clones the repository at `repo_commit`, runs the agent, runs the verification commands).

Failure mode: *If the repository clone fails (network down, repository deleted), the benchmark must record a `Skipped` result (not a crash). The harness does not fail the entire suite because one benchmark is skipped.*

The Benchmark Runner

The harness runs a benchmark in a *single session* (with a *fixed* repository state): 1. *Clone* the repository at `repo_commit`. 2. *Run* the agent (with the prompt, the verification commands). 3. *Collect* the result (pass/fail, steps, time). 4. *Record* the result (to a JSON file).

The harness *does not modify* the original repository (it works in a *temporary clone*). After the benchmark runs, the harness *deletes* the clone (to avoid disk space leaks).

System invariant: *The benchmark runner must isolate each benchmark (a benchmark's file changes must not affect the next benchmark). The harness must delete the cloned repository after each benchmark.*

The Benchmark Suite

A *suite* is a collection of benchmarks (with metadata). The suite file is:

```
name: "coding-agent-harness-benchmarks"
version: "1.0"
benchmarks:
  - "auth-fix-01"
  - "refactor-api-02"
  - "test-cleanup-03"
  - "feature-add-04"
```

The harness *loads* the suite (reads the file), *resolves* each benchmark (loads the JSON file), and *runs* them (sequentially, or in parallel — with a configurable concurrency limit).

The harness *reports* a suite score: - `total` — number of benchmarks. - `passed` — number of benchmarks that passed. - `failed` — number of benchmarks that failed. - `skipped` — number of benchmarks that were skipped (failed to clone, timed out). - `avg_steps` — average step count across *passed* benchmarks.

Qucto in production: Qucto stores benchmark suites in `~/.qucto/benchmarks/*.yaml`. The CLI command `qucto benchmark run suite.yaml` runs the suite (and writes results to `~/.qucto/benchmarks/results/<suite>.json`). The results file contains:

```
{ "benchmarks": { "auth-fix-01": { "outcome": "Complete", "steps": 3, "time_ms": 45000 } }, "score": { "passed": 4, "total": 4, "avg_steps": 3.0 } }
```

.

The Score Format

The score file is a JSON object:

```
{
  "suite_name": "coding-agent-harness-benchmarks",
  "runs_at": "2026-10-02T23:59:00Z",
  "results": {
    "auth-fix-01": {
      "outcome": "Complete",
      "steps": 3,
      "time_ms": 45000,
      "verification_output": "2/2 tests passed"
    }
  },
  "score": {
    "passed": 4,
    "failed": 0,
    "skipped": 0,
    "total": 4,
    "avg_steps": 3.0
  }
}
```

The harness *reports* the score (to the user or the CLI output). The user *can compare* two score files (e.g., "Model A: avg 3.0 steps, 4/4 passed; Model B: avg 5.2 steps, 3/4 passed").

Failure mode: If the score file is corrupted (an incomplete JSON), the harness must not crash. It should log a warning (and skip that score file). The harness does not use a corrupted score for comparison.

Exercise

Write a test that: 1. Creates a benchmark suite with 2 benchmarks (a "pass" and a "fail" benchmark). 2. Runs the suite (with a *mocked* agent that returns "Complete" for the pass benchmark and "VerificationFailed" for the fail benchmark). 3. Verifies that the score file has `passed: 1, failed: 1, total: 2`. 4. Verifies that `avg_steps` is a valid number (not null).

This test demonstrates the *benchmark suite format*: a collection of benchmarks with a *scored* result.

Beyond Rust

Concern	Python	TypeScript
Benchmark file	<code>yaml.safe_load(f)</code>	<code>JSON.parse(fs.readFileSync(...))</code>
Suite score	<code>{passed: 4, total: 4}</code>	Same
Session clone	<code>git clone <url> --depth 1</code>	Same (shell)

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test evaluate suite_reports_score_with_one_passed_one_failed
```

This test verifies that a benchmark suite with two benchmarks (one pass, one fail) reports a score of `{passed: 1, failed: 1, total: 2}`.

13. The Complete Reference Harness

Chapter 13 teaches you to unify all the previous chapters (chapters 1–12) into a complete reference harness. The harness combines the model transport (chapter 1–2), the bounded agent loop (chapter 3), the typed tools

(chapter 4), the execution policy (chapter 5), the verification gate (chapter 6), the session abstraction (chapter 8), the security gates (chapter 9), the MCP expansion (chapter 10), the observability layer (chapter 11), and the evaluation framework (chapter 12) into a single, cohesive application.

The Problem

Chapters 1–12 each teach a *single layer* of the harness. But a *real* harness is *not* a single layer. It is *all layers working together*. The model transport (chapter 1–2) *feeds* the bounded agent loop (chapter 3), which *executes* the typed tools (chapter 4), *gated* by the security policy (chapter 5), *verified* by the completion gate (chapter 6), *isolated* in a session (chapter 8), *scoped* by a profile (chapter 9), *expanded* via MCP (chapter 10), *observed* by telemetry (chapter 11), and *evaluated* by the benchmark suite (chapter 12).

A single-layer harness (e.g., "just the agent loop") *cannot* be run in production. It lacks safety (no policy), lacks verification (no completion gate), lacks isolation (no session), lacks observability (no telemetry), and lacks evaluation (no benchmark).

The *complete reference harness* is *all layers* in a *single application*. It is *tested* (end-to-end, with a real model, a real repository, and a real verification command).

The Application Structure

```
harness/
├── src/
│   ├── lib.rs          (pub mod model, agent, tools, policy, context, session, telemetry, evaluate,
│   ├── model.rs        (HttpModel, parse_assistant, error handling)
│   ├── agent.rs        (Agent::run, RepeatGuard, bounded loop)
│   ├── tools.rs        (Tool trait, ToolRegistry, ReadFile, WriteFile, ApplyPatch, RunCommand)
│   ├── policy.rs       (Decision, Preset, Policy::decide)
│   ├── context.rs      (Context::new, resolve_existing, run_command)
│   ├── session.rs      (Session, Session::push_user/assistant/result)
│   ├── telemetry.rs    (Telemetry::log_event, query)
│   ├── evaluate.rs     (Benchmark, Suite, run_suite)
│   └── mcp.rs          (McpServer, discovery, execute)
├── benchmarks/
└── suite.yaml         (list of benchmarks)
```

```

├── auth-fix-01.json (individual benchmark)
├── ...
└── tests/
    ├── agent_loop.rs (agent loop tests)
    ├── safety.rs (policy + tools + context tests)
    └── mcp.rs (MCP discovery test)

```

The `lib.rs` exports all modules. The `bin/` directory (optional) provides a CLI entry point:

```

harness run --repo <path> --prompt "fix auth" --profile editor
harness benchmark run --suite suite.yaml

```

System invariant: *The CLI interface (if provided) must not expose internal implementation details (the agent's message history, the session's full context). The CLI exposes only the public API (run, benchmark, status). The internal state (session's message list, context's file changes) is not accessible from the CLI.*

The Application Flow

The application *runs* as follows (in a single user invocation): 1. The user *invokes* the CLI: `harness run --repo my-repo/ --prompt "fix auth"`. 2. The harness *loads* the repository (canonicalizes the path). 3. The harness *loads* the profile (from `~/.quecto/profiles/editor.json`, or the default). 4. The harness *loads* the MCP servers (from `~/.quecto/mcp/` — if any are configured). 5. The harness *creates* a session (id = a UUID, root = canonicalized repo path). 6. The harness *pushes* the user's prompt (as a user message). 7. The harness *runs* the agent loop (bounded by steps, with policy gating and MCP tools). 8. The harness *runs* the verification gate (on the session's final changes). 9. The harness *writes* the telemetry (to the log file). 10. The harness *returns* the result (Complete, VerificationFailed, ...).

Failure mode: *If the application fails during step 7 (agent loop), the harness must still write the telemetry (up to that point). The telemetry must contain a `session_end` event with `outcome: "Error"`. The harness must not leave the telemetry incomplete (partial telemetry is harder to debug than no telemetry).*

The Application as a Library

The harness is *also* a library (for embedding in other applications). The `lib.rs` exposes:

```
use harness::{Agent, Session, Policy, Telemetry, Benchmark};

fn main() {
    let policy = Policy::from_preset(Preset::Editor);
    let telemetry = Telemetry::new("/tmp/session-abc.jsonl");
    let mut session = Session::new("/tmp/my-repo");
    session.push_user("fix the auth bug");

    let result = Agent::run(Session {
        session,
        policy,
        telemetry: telemetry.clone(),
        max_steps: 10,
    });

    println!("Outcome: {:?}", result.outcome());
}
```

The library interface *must* be *stable* (semver-major changes are *not* backward-compatible). The library *does not* expose implementation details (e.g., the internal `RepeatGuard` struct is *private*; the `Agent::run` method is *public*).

Quecto in production: Quecto is both a library (embedded in a larger application) and a CLI (for direct use). The library's `pub` interface is the CLI's interface (the CLI wraps the library). If a library method is internal (e.g., `RepeatGuard`), it is not exported (no `pub` in `lib.rs`).

The Application in Production

Quecto runs the harness in *multiple modes*: 1. **CLI** — a single user invocation (a terminal session). The CLI *runs* a single task (or a benchmark suite). 2. **Library** — embedded in a web application (e.g., a dashboard that *offers* coding tasks). The library *exposes* `run()`, `benchmark()`, `status()` (as REST or gRPC endpoints). 3. **CI** — run as a CI step (on a pull request, on a branch merge). The harness *runs* the verification suite (or a set of benchmarks) *in the CI pipeline*.

In all three modes, the harness *behaves identically* (same tools, same policy, same verification, same telemetry). The *only* difference is the *input* (CLI: a prompt string; Library: a Rust function call; CI: a YAML file with a benchmark list).

System invariant: *The harness must produce the same result (for the same prompt, same repository, same model) regardless of the mode (CLI, Library, CI). If a CLI invocation returns `VerificationFailed`, a Library invocation with the same inputs must also return `VerificationFailed` (modulo non-deterministic factors: a test suite with a flaky test, a network timeout on a model API).*

The Application as a Teaching Tool

The complete reference harness *is also* the *teaching material* for this book. The book *uses* the application's source code (via `{{include:}}` directives) to *show* the implementation of each layer (chapters 1–12). The book *does not* abstract away the source code (the reader sees the actual Rust structs and methods).

This is *intentional*. The book's goal is *not* to teach *abstract principles* ("safety is important"). The book's goal is to teach *concrete implementation* (a `Policy::decide` method that returns `Deny`, a `Context::resolve_existing` that *rejects* path escapes).

Failure mode: *If the book discrepancies the source code (e.g., the book says "the policy rejects all commands" but the source code says `Policy::decide` only rejects `sudo` and `git push`), the reader is misled. The book must always reflect the current source code. The book must update its `{{include:}}` directives when the source code changes.*

Exercise

Write a test that: 1. Creates a *complete* harness (with all modules: model, agent, tools, policy, context, session, telemetry). 2. Runs the harness with a *mocked* model (returning one `WriteFile` call and then

"done"). 3. Verifies that: - The session's `result` is `Outcome::Complete`. - The telemetry file contains 5 events (`session_start`, `tool_call`, `tool_result`, `session_end`, `error` or ... — depending on the mock's output). - The verification gate *ran* (and returned `true` if the verification command is a no-op).

This test demonstrates the *complete application*: every layer (model, agent, tools, policy, context, session, telemetry, verification) *works together* in a single invocation.

Beyond Rust

Concern	Python	TypeScript
Library import	<code>from harness import Agent, Session</code>	<code>import { Agent, Session } from 'harness'</code>
CLI invocation	<code>harness run --repo path</code>	Same (Node's <code>child_process.spawn</code>)
CI mode	<code>harness benchmark run suite.yaml</code>	Same (shell)

Build checkpoint

```
cd books/building-a-coding-agent-harness/examples/reference-harness
cargo test --test agent_loop complete_harness_integration_with_telemetry_and_verification
```

This test verifies that a full harness (with telemetry, session, policy, verification) runs one task, records 5 events in the log, and returns `Outcome::Complete`.

Appendix A. The Quecto Reference

This appendix maps every chapter of the book to the actual production code in Quecto (the product that inspired this book). The appendix is not theoretical. It pointers you to the real source code (in the `quecto` repository) that implements each chapter's lesson.

Mapping the Chapters

Chapter	Title	Quecto Module	Module Path	Key Types
1	The Harness Is the System	<code>quecto_core</code>	<code>src/quecto_core/</code>	<code>Quecto::new()</code> , <code>Quecto::run()</code>
2	One Model Call, No Framework	<code>quecto_core::model</code>	<code>src/quecto_core/model/</code>	<code>HttpModel</code> , <code>ModelError</code>
3	The Bounded Agent Loop	<code>quecto_core::agent</code>	<code>src/quecto_core/agent/</code>	<code>Agent::run</code> , <code>Outcome</code> , <code>RepeatGuard</code>
4	Typed, Unsafe-Free Communication	<code>quecto_core::tools</code>	<code>src/quecto_core/tools/</code>	<code>Tool</code> , <code>ToolRegistry</code> , <code>RunCommand</code>
5	Execution Policy and Approval Gates	<code>quecto_core::policy</code>	<code>src/quecto_core/policy/</code>	<code>Policy::decide</code> , <code>Decision</code>
6	Verification as a Completion Gate	<code>quecto_core::verify</code>	<code>src/quecto_core/verify/</code>	<code>Verifier::check</code> , <code>VerificationResult</code>
7	Context and Instruction Loading	<code>quecto_core::context</code>	<code>src/quecto_core/context/</code>	<code>Context</code> , <code>resolve_existing</code>
8	Sessions and State Management	<code>quecto_core::session</code>	<code>src/quecto_core/session/</code>	<code>Session</code> , <code>Session::push</code>

Chapter	Title	Qecto Module	Module Path	Key Types
9	Profiles and Security Gates	<code>qecto_core::profile</code>	<code>src/qecto_core/profile/</code>	<code>Profile</code> , <code>Profile::from_json</code>
10	MCP — Model Context Protocol	<code>qecto_mcp</code>	<code>crates/qecto_mcp/</code>	<code>McpServer</code> , <code>McpError</code>
11	Observability and Telemetry	<code>qecto_telemetry</code>	<code>crates/qecto_telemetry/</code>	<code>Telemetry</code> , <code>Event</code>
12	Evaluation and Benchmarks	<code>qecto_evaluate</code>	<code>crates/qecto_evaluate/</code>	<code>Benchmark</code> , <code>Suite</code>
13	The Complete Reference Harness	<code>qecto_cli</code>	<code>bin/qecto</code>	<code>harness run</code> , <code>harness benchmark</code>

How to Read the Qecto Code

For each chapter, *open* the corresponding module (in the `qecto` repository) and *read* the public API (the `pub` structs and methods). Do *not* read the *internal* implementation (the `fn` methods that are *not* `pub`). The public API *is* what this book teaches. The internal implementation *is* implementation detail.

For example, for Chapter 4 (typed tools), *open* `src/qecto_core/tools/mod.rs` and *read* the `pub trait Tool` definition. Do *not* read the `impl ReadFile` method (unless you want to understand the *internal* file I/O).

Learning strategy: Read the book's source code (in this worktree: `examples/reference-harness/src/`) first. Then open the corresponding module in the `qecto` repository. Compare them (what is the same? what is different?). The book's source is simplified; the Qecto source is production (with error handling, edge cases, and performance considerations).

The Qecto vs. This Book

Aspect	This Book	Qecto (production)
Tools		+ Grep, GitStatus, ListFiles, Python, Bash

Aspect	This Book	Quecto (production)
	ReadFile, WriteFile, ApplyPatch, RunCommand	
Policy	3 presets (ReadOnly, Editor, Full)	Profiles (named bundles, per-repo, sub-profiles)
Security	Hard-denied command patterns (substring)	Token-based matching (first token in command)
Session	Single message list	Per-session file diff + verification state + retry count
MCP	stdio protocol (JSON-RPC 2.0)	stdio + HTTP (server can run remotely)
Telemetry	Local JSONL file	Local JSONL + remote server (POST)
Evaluation	<code>Benchmark</code> + <code>Suite</code> structs	CLI (<code>quecto benchmark run</code>) + web dashboard

Important: The book intentionally simplifies the Quecto reference. The book teaches concepts (the harness layers). The Quecto source implements those concepts with production concerns (error handling, concurrent safety, performance). Do not think the book is "wrong" because the Quecto source is more complex. The book is foundational; the Quecto source is applied.

The Quecto Command Reference

The `quecto` CLI (the *production* command-line tool) maps *directly* to the chapters:

CLI Command	Chapter
<code>quecto run --repo <path></code>	Chapters 3, 13
<code>quecto run --profile <name></code>	Chapter 9
<code>quecto run --prompt "fix ..." --verify "cargo test"</code>	Chapter 6
<code>quecto run --mcp server.json</code>	Chapter 10
<code>quecto logs --session <id></code>	Chapter 11
<code>quecto benchmark run suite.yaml</code>	Chapter 12
<code>quecto status</code>	Chapter 13 (post-run status)

Each command *prints* (to the terminal) the *same data* that the CLI library returns (the `Outcome` enum, the telemetry events, the score). The CLI *is the presentation layer* (over the library).

Quecto in production: The CLI is two-layer: (1) a library (`lib.rs`), (2) a binary (`bin/quecto`). The binary calls the library (it does not duplicate the library's logic). The binary's only job is to parse CLI arguments, call the library, and print the result (formatted for the terminal).

Appendix B. Further Reading

1. **Anthropic API (Messages API)** — *how the model API works.*
<https://docs.anthropic.com/en/api/messages>
2. **OpenAI Tool Calling** — *how tool calling works in the OpenAI API.*
<https://platform.openai.com/docs/guides/function-calling>
3. **JSON-RPC 2.0 Specification** — *the protocol that MCP uses.*
<https://www.jsonrpc.org/specification>
4. **git sparse-checkout** — *if you clone the full Quecto repository, use sparse-checkout to avoid downloading all modules.* `git sparse-checkout set quecto_core`
5. **cargo doc (Rust documentation)** — *to browse the Quecto source.* `cargo doc --open --package quecto_core`

Appendix C. Glossary

Term	Definition
Harness	The <i>system boundary</i> (model vs. harness). The harness is the <i>infrastructure</i> that sits <i>between</i> the model and the user.
Policy	A <i>gate</i> that decides whether a tool call should <i>execute</i> . Returns <code>Allow</code> , <code>Ask</code> , or <code>Deny</code> .
Session	A <i>single coding task's</i> isolated state (message history, file changes, verification result).
Profile	

Term	Definition
	A <i>named configuration</i> (like "editor" or "docker-build") that selects <i>which</i> tools are allowed, <i>which</i> commands are blocked.
MCP (Model Context Protocol)	A <i>protocol</i> for the harness to <i>discover</i> tools (at startup) and <i>execute</i> tools (over a protocol) without hard-coding the tools' source.
Telemetry	<i>Observability</i> : the logging of every visible event (tool calls, errors, verification outcomes) to a durable file.
Benchmark	A <i>defined</i> coding task (repository, prompt, verification commands) with a <i>measurable</i> outcome (pass/fail).
ReviewGate	(From earlier chapters, not in this book) A <i>post-edit</i> review step (before the changes are applied) that checks the diff for <i>obvious regressions</i> (deleted imports, removed error handling). <i>Not included in this book (intentionally out of scope)</i> .
Progressive Disclosure	A <i>UI/UX principle</i> : show the <i>simple</i> configuration by default; show the <i>advanced</i> configuration only when the user <i>opts in</i> . (Quecto's approach: default <code>ReadOnly</code> ; user <i>explicitly</i> switches to <code>Full</code> .)

Appendix D. Build the Book (PDF)

To generate a PDF from this manuscript (if you want a *printable* version of this book):

1. Ensure `build_book.py` (and the `pdf/` modules) are installed and runnable (see the book's `build/` directory).
2. Run `python3 build_book.py --output static/books/building-a-coding-agent-harness.pdf`.
3. Validate the PDF (page count 70–80, TOC, bookmarks, no clipping).

Note: The `build_book.py` and `pdf/` modules are not included in this book's manuscript. They are in the worktree (at `examples/build/` and `examples/pdf/`). The PDF pipeline is a separate project (a Python script using `reportlab` or `weasyprint`). If you want to produce a PDF, the pipeline code is in that directory.

End of Appendix A.

Appendix A. The Quecto Reference

This appendix maps every chapter of the book to the actual production code in Quecto (the product that inspired this book). The appendix is not theoretical. It pointers you to the real source code (in the `quecto` repository) that implements each chapter's lesson.

Mapping the Chapters

Chapter	Title	Quecto Module	Module Path	Key Types
1	The Harness Is the System	<code>quecto_core</code>	<code>src/quecto_core/</code>	<code>Quecto::new()</code> , <code>Quecto::run()</code>
2	One Model Call, No Framework	<code>quecto_core::model</code>	<code>src/quecto_core/model/</code>	<code>HttpModel</code> , <code>ModelError</code>
3	The Bounded Agent Loop	<code>quecto_core::agent</code>	<code>src/quecto_core/agent/</code>	<code>Agent::run</code> , <code>Outcome</code> , <code>RepeatGuard</code>
4	Typed, Unsafe-Free Communication	<code>quecto_core::tools</code>	<code>src/quecto_core/tools/</code>	<code>Tool</code> , <code>ToolRegistry</code> , <code>RunCommand</code>
5	Execution Policy and Approval Gates	<code>quecto_core::policy</code>	<code>src/quecto_core/policy/</code>	<code>Policy::decide</code> , <code>Decision</code>
6	Verification as a Completion Gate	<code>quecto_core::verify</code>	<code>src/quecto_core/verify/</code>	<code>Verifier::check</code> , <code>VerificationResult</code>
7	Context and Instruction Loading	<code>quecto_core::context</code>	<code>src/quecto_core/context/</code>	<code>Context</code> , <code>resolve_existing</code>
8		<code>quecto_core::session</code>	<code>src/quecto_core/session/</code>	<code>Session</code> , <code>Session::push</code>

Chapter	Title	Qeucto Module	Module Path	Key Types
	Sessions and State Management			
9	Profiles and Security Gates	<code>quecto_core::profile</code>	<code>src/quecto_core/profile/</code>	<code>Profile</code> , <code>Profile::from_json</code>
10	MCP — Model Context Protocol	<code>quecto_mcp</code>	<code>crates/quecto_mcp/</code>	<code>McpServer</code> , <code>McpError</code>
11	Observability and Telemetry	<code>quecto_telemetry</code>	<code>crates/quecto_telemetry/</code>	<code>Telemetry</code> , <code>Event</code>
12	Evaluation and Benchmarks	<code>quecto_evaluate</code>	<code>crates/quecto_evaluate/</code>	<code>Benchmark</code> , <code>Suite</code>
13	The Complete Reference Harness	<code>quecto_cli</code>	<code>bin/quecto</code>	<code>harness run</code> , <code>harness benchmark</code>

How to Read the Qeucto Code

For each chapter, *open* the corresponding module (in the `quecto` repository) and *read* the public API (the `pub` structs and methods). Do *not* read the *internal* implementation (the `fn` methods that are *not* `pub`). The public API *is* what this book teaches. The internal implementation *is* implementation detail.

For example, for Chapter 4 (typed tools), *open* `src/quecto_core/tools/mod.rs` and *read* the `pub trait Tool` definition. Do *not* read the `impl ReadFile` method (unless you want to understand the *internal* file I/O).

Learning strategy: Read the book's source code (in this worktree: `examples/reference-harness/src/`) first. Then open the corresponding module in the `quecto` repository. Compare them (what is the same? what is different?). The book's source is simplified; the Qeucto source is production (with error handling, edge cases, and performance considerations).

The Quecto vs. This Book

Aspect	This Book	Quecto (production)
Tools	ReadFile, WriteFile, ApplyPatch, RunCommand	+ Grep, GitStatus, ListFiles, Python, Bash
Policy	3 presets (ReadOnly, Editor, Full)	Profiles (named bundles, per-repo, sub-profiles)
Security	Hard-denied command patterns (substring)	Token-based matching (first token in command)
Session	Single message list	Per-session file diff + verification state + retry count
MCP	stdio protocol (JSON-RPC 2.0)	stdio + HTTP (server can run remotely)
Telemetry	Local JSONL file	Local JSONL + remote server (POST)
Evaluation	Benchmark + Suite structs	CLI (<code>quecto benchmark run</code>) + web dashboard

Important: The book intentionally simplifies the Quecto reference. The book teaches concepts (the harness layers). The Quecto source implements those concepts with production concerns (error handling, concurrent safety, performance). Do not think the book is "wrong" because the Quecto source is more complex. The book is foundational; the Quecto source is applied.

The Quecto Command Reference

The `quecto` CLI (the *production* command-line tool) maps *directly* to the chapters:

CLI Command	Chapter
<code>quecto run --repo <path></code>	Chapters 3, 13
<code>quecto run --profile <name></code>	Chapter 9
<code>quecto run --prompt "fix ..." --verify "cargo test"</code>	Chapter 6
<code>quecto run --mcp server.json</code>	Chapter 10
<code>quecto logs --session <id></code>	Chapter 11
<code>quecto benchmark run suite.yaml</code>	Chapter 12

Each command *prints* (to the terminal) the *same data* that the CLI library returns (the `Outcome` enum, the telemetry events, the score). The CLI *is the presentation layer* (over the library).

Quecto in production: The CLI is two-layer: (1) a library (`lib.rs`), (2) a binary (`bin/quecto`). The binary calls the library (it does not duplicate the library's logic). The binary's only job is to parse CLI arguments, call the library, and print the result (formatted for the terminal).

Appendix B. Further Reading

1. **Anthropic API (Messages API)** — *how the model API works.*
<https://docs.anthropic.com/en/api/messages>
2. **OpenAI Tool Calling** — *how tool calling works in the OpenAI API.*
<https://platform.openai.com/docs/guides/function-calling>
3. **JSON-RPC 2.0 Specification** — *the protocol that MCP uses.*
<https://www.jsonrpc.org/specification>
4. **git sparse-checkout** — *if you clone the full Quecto repository, use sparse-checkout to avoid downloading all modules.* `git sparse-checkout set quecto_core`
5. **cargo doc (Rust documentation)** — *to browse the Quecto source.* `cargo doc --open --package quecto_core`

Appendix C. Glossary

Term	Definition
Harness	The <i>system boundary</i> (model vs. harness). The harness is the <i>infrastructure</i> that sits <i>between</i> the model and the user.
Policy	A <i>gate</i> that decides whether a tool call should <i>execute</i> . Returns <code>Allow</code> , <code>Ask</code> , or <code>Deny</code> .

Term	Definition
Session	A single coding task's isolated state (message history, file changes, verification result).
Profile	A named configuration (like "editor" or "docker-build") that selects <i>which</i> tools are allowed, <i>which</i> commands are blocked.
MCP (Model Context Protocol)	A protocol for the harness to <i>discover</i> tools (at startup) and <i>execute</i> tools (over a protocol) without hard-coding the tools' source.
Telemetry	<i>Observability</i> : the logging of every visible event (tool calls, errors, verification outcomes) to a durable file.
Benchmark	A <i>defined</i> coding task (repository, prompt, verification commands) with a <i>measurable</i> outcome (pass/fail).
ReviewGate	(From earlier chapters, not in this book) A <i>post-edit</i> review step (before the changes are applied) that checks the diff for <i>obvious regressions</i> (deleted imports, removed error handling). <i>Not included in this book (intentionally out of scope)</i> .
Progressive Disclosure	A UI/UX principle: show the <i>simple</i> configuration by default; show the <i>advanced</i> configuration only when the user <i>opts in</i> . (Quecto's approach: default <code>ReadOnly</code> ; user <i>explicitly</i> switches to <code>Full</code> .)

Appendix D. Build the Book (PDF)

To generate a PDF from this manuscript (if you want a *printable* version of this book):

1. Ensure `build_book.py` (and the `pdf/` modules) are installed and runnable (see the book's `build/` directory).
2. Run `python3 build_book.py --output static/books/building-a-coding-agent-harness.pdf`.
3. Validate the PDF (page count 70–80, TOC, bookmarks, no clipping).

Note: The `build_book.py` and `pdf/` modules are not included in this book's manuscript. They are in the worktree (at `examples/build/` and `examples/pdf/`). The PDF pipeline is a separate project (a Python script using `reportlab` or `weasyprint`). If you want to produce a PDF, the pipeline code is in that directory.

End of Appendix A.